

VSGM: View-Based GPU-Accelerated Subgraph Matching on Large Graphs

Guanxian Jiang, Qihui Zhou, Tatiana Jin, Boyang Li, Yunjian Zhao, Yichao Li, James Cheng

The Chinese University of Hong Kong

Hong Kong, China

{gxjiang, qhzhou, tjin, byli, yjzhao, yichaoli21, jcheng}@cse.cuhk.edu.hk

Abstract—Subgraph matching is a fundamental building block in graph analytics. Due to its high time complexity, GPU-based solutions have been proposed for subgraph matching. Most existing GPU-based works can only cope with relatively small graphs that fit in GPU memory. To support efficient subgraph matching on large graphs, we propose a view-based method to hide communication overhead and improve GPU utilization. We develop VSGM, a subgraph matching framework that supports efficient pipelined execution and multi-GPU architecture. Extensive experimental evaluation shows that VSGM significantly outperforms the state-of-the-art solutions.

Index Terms—subgraph matching, GPU acceleration

I. INTRODUCTION

Subgraph matching is one of the most fundamental graph problems and has a broad range of applications [1] such as social network analysis [2], finance [3], bio-informatics [4], chemistry [5] and so on. Given a data graph G and a query graph P , subgraph matching finds all subgraphs of G that are isomorphic to P . Subgraph matching is computation intensive, and hence many GPU-based solutions have been proposed to leverage the massive parallelism of GPU to improve its performance [6]–[11]. However, most of these works assume that the data graph can fit in the memory of a GPU, and thus they cannot handle many large graphs (especially those in industry) that easily exceed the memory capacity of a GPU.

A recent work, called PBE [10], supports subgraph matching on large graphs by partitioning a data graph, so that each partition fits in GPU memory. The matching processing is split into intra-partition search and inter-partition search. Intra-partition search is simple and efficient as it only finds matches within a partition. However, inter-partition search requires to repeatedly load the cross-partition neighbors of the candidate vertices on demand for matching. GPU threads are idle when waiting for the required data to be loaded from main memory of the host machine into GPU memory. A possible solution is caching, but highly frequent exchanges within the cache may occur and render a marginal performance improvement.

We propose a view-based method to eliminate on-demand data loading during subgraph matching. The method defines a set of **view bins**. Each bin consists of the views that contain all the necessary data for subgraph matching starting from a set of source vertices, where the sets of source vertices in all bins are disjoint, and their union forms the vertex set of the data graph. We analyze input query patterns to deduce the required

information for constructing the view bins for their matching. To save space and reduce data transmission volume, we pack the views of multiple source vertices into a bin, while ensuring that each bin can fit in GPU memory. We model our problem as a variant of the NP-hard bin packing problem [12], [13] and propose an efficient K-Means-based heuristic approach to reduce the number of view bins.

We then design and implement a **View-based SubGraph Matching (VSGM)** framework. Given a query pattern, VSGM first considers different match orders and decides the view bins needed for the pattern, with which it constructs an execution plan and creates the view bins on-the-fly for matching the query. We adopt the Multi-Producer-Multi-Consumer (MPMC) design to manage the buffer of view bins and to overlap data transfer and GPU computation. In addition, VSGM also supports a multi-GPU architecture to process large graphs or complex patterns. We verify by experiments that VSGM can handle large graphs with limited GPU memory and it significantly outperforms the state-of-the-art GPU-based solution [10].

II. BACKGROUND AND MOTIVATION

A **data graph** is defined by $G = (V_G, E_G)$, where V_G and E_G are the set of vertices and edges in G . A **query graph**, also called a **pattern**, is defined by $P = (V_P, E_P)$. Both data graph and query graph are undirected, unlabeled and simple (i.e., no self-loops and multiple edges) in this work. The neighbor set of a vertex v in a graph is denoted by $N(v)$, i.e., for each $u \in N(v)$, the edge (u, v) exists in the graph.

A subgraph H of G is a **match** of P if H is **isomorphic** to P , meaning that there exists a bijective mapping $f : V_P \rightarrow V_H$, such that $(u_1, u_2) \in E_P$ if and only if $(f(u_1), f(u_2)) \in E_H$. Given a data graph G and a pattern P , **subgraph matching** is to find all matches (or instances) of P in G . The matching of P follows a **match order**, i.e., an order to map the vertices of P to vertices in G . A match order is essentially a permutation of the vertices in P . We use Π_P to denote the set of all match orders of P . For efficient subgraph matching, it is generally required that for each $\pi \in \Pi_P$, the subgraph of P induced by a prefix of π is a connected subgraph. The set of match orders starting from a vertex u in P is denoted as $\Pi_P(u)$ and we have $\Pi_P(u) \subseteq \Pi_P$.

A. GPU-Based Subgraph Matching

Many GPU-based solutions have been proposed for subgraph matching. GPSM [7] adopts a filtering-and-joining approach to leverage the power of massive parallelism. Each warp is assigned to process a candidate vertex in the filtering phase as well as a partial match in the joining phase. It achieves good coalesced memory access and gains significant speedup compared to CPU-based solutions. GSI [9] found that the edge-based join operation in GPSM requires a two-step output scheme [14], which conducts the same join operation twice. GSI prevents this using vertex-oriented join. It also proposes a data structure, called Partitioned Compressed Sparse Row, to reduce the access latency of GPU memory. GunrockSM [8] uses a filtering-and-verification strategy built on a graph analytic framework called Gunrock [15]. Compared to the above works which follows a BFS strategy, a recent work cuTS [11] proposes a hybrid BFS-DFS strategy to limit memory usage when enabling high parallelism.

One key limitation of the above works is that they all assume the entire data graph can fit in the memory of a GPU. To overcome this limitation, PBE [10] proposes partition-based enumeration to divide a data graph into multiple partitions and ensure that each partition (only with intra-partition edges) fits in GPU memory. The matching process is separated into **intra-partition search** and **inter-partition search**. In intra-partition search, it only loads each partition excluding all out-going edges into GPU memory and searches all matches within the partition. Thus, no access to the data graph in main memory of the host machine is needed. Inter-partition search enumerates all cross-partition edges to find all the remaining matches, which brings redundant computation as a cross-partition edge can be mapped into two vertices of the pattern with multiple match orders simultaneously. The redundant computation also causes higher PCI-e communication overhead as it needs to fetch the data graph from the host memory. PBE uses shared-execution to group different match orders together so that fewer match plans are generated by combining strict-equivalent groups of match orders. It also conducts the matching by extending each inter-partition edge in one group of match orders at a time. Thus, redundant computation is reduced. However, there is still much **on-demand data loading**, since PBE needs to repeatedly load the cross-partition neighbors of the candidate vertices from the host memory to GPU memory for computing the matches, as our profiling results also show below.

B. An Analysis of the PBE Solution

As PBE is the only GPU-based solution for subgraph matching in large graphs that cannot fit in GPU memory, we conduct a detailed analysis on PBE to motivate our work. Table I shows some profiling results of PBE using the same experiment setting as in Section V-B, where *Execution Time (%)* lists the percentage of the total time, *GPU Utilization (%)* are the average usage of GPU, and *Host-to-Device Transfer* shows the total amount of data transferred from the host memory to GPU

TABLE I: Profiling results of PBE on FR graph using 1 GPU

Pattern	Execution Time (%)		GPU Utilization (%)			Host-to-Device Transfer
	Intra	Inter	Intra	Inter	Overall	Size Ratio
3-clique	9.3%	90.7%	89%	17%	24%	$\approx 35.3\times$
4-clique	12.6%	87.4%	94%	19%	28%	$\approx 77.1\times$
4-diamond	4.3%	95.7%	91%	18%	21%	$\approx 134.0\times$

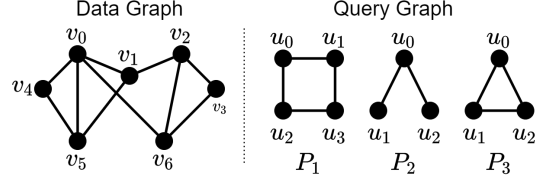


Fig. 1: An example data graph and three query graphs

memory divided by the size of the data graph itself (stored in the CSR format in the host memory).

The results show that the majority of the time is spent on inter-partition search, but its GPU utilization is less than 19% (and as a result, the overall GPU utilization is also low). On the contrary, intra-partition search has high GPU utilization, which allows its execution to be completed much faster. This is because inter-partition search has much on-demand data loading, during which the GPU threads are idle waiting for the data needed for the computation. As the last column shows, the total amount of data transferred due to on-demand data loading is from 35.3 to 134.0 times of the size of the data graph. This is a high overhead, which also shows that there is great room for improvement and hence motivates our work.

III. VIEW-BASED GRAPH PREPARATION

This section discusses how to prepare the necessary data (e.g., views) and minimize data transfer to GPUs for subgraph matching.

A. A Motivating Example

The problem of on-demand graph data loading makes us rethink the neighbor accessing behavior for matching query patterns. The subgraph matching process starts from each vertex in the data graph as the source vertex and iteratively expands it by accessing the source vertex's multi-hop neighbors to find matches. Thus, we can completely avoid on-demand data loading by preparing the multi-hop neighbors for each source vertex.

Example 1. Consider the data graph G and the first query P_1 in Figure 1. Assume that we use a match order $\pi_1 = \{u_0, u_1, u_2, u_3\}$ that starts from u_0 . Then all the matches starting from a source vertex $v \in V_G$ can be found by accessing only the 2-hop neighbors of v , because matching both u_1 and u_2 requires only the 1-hop neighbors of v , and matching u_3 only accesses 1 more hop from the vertices that match u_1 and u_2 . In comparison, another match order $\pi_2 = \{u_0, u_1, u_3, u_2\}$ leads to traverse 3 hops from a source vertex v : first, to match u_1 , we need the 1-hop neighbors of

v ; then to match u_3 , we need the 1-hop neighbors of those vertices that match u_1 ; and finally to match u_2 , we need the intersection of the 1-hop neighbors of v and the 1-hop neighbors of those vertices that match u_3 , where the latter are the 3-hop neighbors of v .

To generalize the example to an arbitrary match order $\pi = \{u'_0, u'_1, \dots, u'_{|P|-1}\}$, we define **vertex level** and **edge level** to identify how many hops from a source vertex should be visited in order to match P by following π . The level of the source vertex u'_0 is $l_{u'_0} = 0$. For any other vertex u'_i , its level $l_{u'_i}$ is the shortest-path distance between u'_0 and u'_i in the subgraph of P induced by $\{u'_0, u'_1, \dots, u'_i\}$, or formally $l_{u'_i} = \min_{j < i, u'_j \in N(u'_i)} l_{u'_j} + 1$. The level of an edge (u'_i, u'_j) , where $j < i$, is defined as $l_{(u'_i, u'_j)} = l_{u'_j} + 1$. Thus, the **maximum edge level** indicates the number of hops that is needed to match P by following π . Using the example of π_2 above, $l_{u_1} = 1, l_{u_3} = 2$, and the edge (u_3, u_2) has the greatest level $l(u_3, u_2) = l_{u_3} + 1 = 3$.

B. View Construction

The above example motivates us to use views, i.e., multi-hop neighbors, to avoid on-demand data loading. We first define view as follows. Given a data graph G , the **k -hop neighbors** of a vertex $v \in V_G$, denoted by $N_{v,k}$, is given by:

$$N_{v,k} = \begin{cases} \{v\} & , k = 0 \\ N_{v,k-1} \cup \left(\bigcup_{u \in N_{v,k-1}} N(u) \right) & , k > 0 \end{cases} \quad (1)$$

Following Equation (1), we construct the **k -hop view** of v , denoted by $S_{v,k}$, by visiting the $(k-1)$ -hop neighbors of v and grouping their adjacency lists (i.e., 1-hop neighbors) in the CSR format. The size of the CSR format of $S_{v,k}$, denoted by $|S_{v,k}|$, is $\sum_{u \in N_{v,k-1}} |N(u)|$.

Lemma III.1. *Given a data graph G , all the matches of a pattern P starting from a source vertex $v \in V_G$ can be obtained from $S_{v,k}$, if there exists a match order $\pi \in \Pi_P$ for which the maximum edge level l_{max} is no greater than k .*

Proof. We first show that all the matches of any vertex $u_i \in V_P$ are in $S_{v,l_{max}}$. Consider the prefix $\pi_i = \{u_0, u_1, \dots, u_i\}$ of π and the subgraph P_i of P induced by π_i . As l_{u_i} is the shortest-path distance between u_0 and u_i in P_i , any match of u_i must be within $l_{u_i} \leq l_{max}$ hops from v (as v matches u_0). Next, to match any query edge (u_j, u_i) , where u_j comes before u_i in π , the 1-hop neighbors of the vertices matching u_j need to be accessed, which are within $l_{u_j} + 1$ hops from v . Since $l_{(u_j, u_i)} = l_{u_j} + 1 \leq l_{max}$, the vertices matching u_j must be in $S_{v,l_{max}-1}$ and thus their 1-hop neighbor information must be contained in $S_{v,l_{max}}$. $\square$

Example 2. Consider Figure 1, assume that v_4 is the source vertex, we have $N_{v_4,0} = \{v_4\}$ and $N_{v_4,1} = \{v_0, v_4, v_5\}$. To construct $S_{v_4,2}$, we group the adjacency lists of the vertices in $N_{v_4,1}$, i.e., $S_{v_4,2} = \{N(v_0), N(v_4), N(v_5)\}$. We can use $S_{v_4,2}$

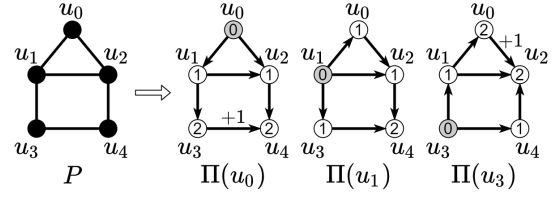


Fig. 2: An illustration of finding the minimum k

to find all matches of P_2 and P_3 starting from v_4 following any match order. We can also use $S_{v_4,2}$ to find all matches of P_1 following π_1 in Example 1, but not π_2 since the maximum edge level for π_2 is 3 (e.g., we may match u_0 with v_4 , u_1 with v_0 and u_3 with v_1 , but we cannot further match u_2 without the neighbors of v_1 because $N(v_1)$ is not in $S_{v_4,2}$).

Minimum k for k -hop views. Different match orders may be used to match a pattern and the choice of a match order affects the hop number k for k -hop views. To reduce the view sizes, we want to use the match order that gives the smallest k .

To efficiently find match order(s) with the smallest k , we narrow down the search space by grouping the match orders into $|V_P|$ groups according to their starting vertex, i.e., $\Pi_P(u)$ for each $u \in V_P$. We first find the minimum hop value k_{min} for each group $\Pi_P(u)$ and then pick the minimum k_{min} among all groups for view generation. Let $l_{max}(\pi_i)$ be the maximum edge level for $\pi_i \in \Pi_P(u)$. Thus $k_{min} = \min_{\pi_i \in \Pi_P(u)} l_{max}(\pi_i)$ and $S_{v,k_{min}}$ is the smallest k -hop view for matching P starting from v , where v matches u . For example, for P_2 in Figure 1, $k_{min} = 1$ for $\Pi_{P_2}(u_0)$ and $k_{min} = 2$ for $\Pi_{P_2}(u_1)$ and $\Pi_{P_2}(u_2)$. Thus, we should pick the minimum $k_{min} = 1$ and construct 1-hop views to match P_2 following a match order in $\Pi_{P_2}(u_0)$.

To compute k_{min} for each $\Pi_P(u)$, we can conduct a BFS on P starting from source u and construct a BFS tree with u as the root. Thus, u is at level 0 of the BFS tree and for each v at level i , i is the shortest-path distance between u and v in P , i.e., i is the minimum $l(v)$ among all $\pi \in \Pi_P(u)$. To obtain k_{min} , we only need to check the vertices at the lowest level, say level l : if an edge exists between any pair of vertices at level l , $k_{min} = l + 1$ because we need 1 more hop to cover such edges; otherwise, $k_{min} = l$. Finally, we pick the minimum k_{min} among the k_{min} for all $\Pi_P(u) \subset \Pi_P$.

The above algorithm also allows us to select a good match order for matching P . We pick the group $\Pi_P(u)$ that gives the minimum k_{min} and construct a match order as follows. The vertices in the match order are first ordered in ascending order of their level in the BFS tree, and then the vertices at the same level are ordered in descending order of their degree for better pruning effects.

Example 3. Figure 2 gives an illustration of finding the minimum k for a 5-node pattern P . Consider three groups of match orders: $\Pi(u_0)$, $\Pi(u_1)$ and $\Pi(u_3)$. The numbers in

the circles show the levels of the vertices in their respective BFS trees. The lowest level l for all the three groups is 2. For $\Pi(u_0)$, there is one edge (u_3, u_4) at level l and thus its $k_{\min} = l + 1 = 3$. Similarly, $k_{\min}$ for $\Pi(u_3)$ is also 3 because of the edge (u_0, u_2) at level l . For $\Pi(u_1)$, only u_4 is at level l and thus its $k_{\min} = l = 2$. Based on the BFS tree of $\Pi(u_1)$, we construct a match order $\{u_1, u_2, u_0, u_3, u_4\}$.

C. View Bin Packing

Although we find the minimum k for constructing the k -hop views, the PCI-e communication cost for processing a query pattern P is still very high if we individually send views of all source vertices to GPU for processing. One straightforward idea is to pack multiple views together. That is, we assign all source vertices in the data graph to disjoint sets and pack the views for each set of source vertices into a bin. The objective is to minimize the number of bins under the bin size constraint due to limited GPU memory capacity. As discussed in Section III-B, the union operation in Equation (1) enables efficient construction of views in the CSR format. Similarly, it also enables efficient construction of bins by grouping the adjacency lists of the vertices within $(k - 1)$ hops of any involved source vertex v . As $|E_G|$ is typically much larger than $|V_G|$ for large real-world graphs, we measure the bin size by the total size of the adjacency lists in each bin. We formally define the view bin packing problem as follows.

Definition 1. Given a vertex set V_G , a bin capacity $M \in \mathbb{Z}^+$, and a function $\text{size}(V) \in \mathbb{Z}^+$ that maps a set $V \subseteq V_G$ of vertices to the space needed to pack their views, the **view bin packing problem** is to partition V_G into disjoint sets $V_1, \dots, V_b$ such that b is minimized under the constraint that $\text{size}(V_i) \leq M$ for each V_i , where $1 \leq i \leq b$.

Lemma III.2. The view bin packing problem is NP-Hard.

Proof. The bin packing problem [12], [13], which considers $\text{size}(\{v_i, v_j\}) = \text{size}(\{v_i\}) + \text{size}(\{v_j\})$ for any pair of vertices v_i and v_j , is known to be NP-hard. In our view bin packing problem, two source vertices v_i and v_j in the same bin may share some $(k - 1)$ -hop neighbors, i.e., $\text{size}(\{v_i, v_j\}) \leq \text{size}(\{v_i\}) + \text{size}(\{v_j\})$. Thus, it is a variant of the bin packing problem that has been proven to be NP-Hard [16]. $\square$

1) *Next-Fit Bin Packing:* Due to the high complexity of view bin packing, we resort to using efficient heuristics to find a good solution. Intuitively, a good heuristic should pack vertices so that their $(k - 1)$ -hop neighbors overlap as much as possible. We first present an efficient heuristic, the next-fit algorithm, to illustrate view bin packing. Next-fit packs vertices one at a time following their order in the data graph (e.g., ordered by vertex IDs), as shown in Example 4.

Example 4. Figure 3 presents the data graph in Figure 1 as a set V_G of vertices and their adjacency lists. Consider view bin packing for 2-hop views and $M = 15$. Next-fit first puts v_0 into a new bin, which gives a bin size of $\text{size}(\{v_0\}) = |S_{v_0,2}| = |N_{v_0}| + |N_{v_1}| + |N_{v_4}| + |N_{v_5}| + |N_{v_6}| = 15$. Next,

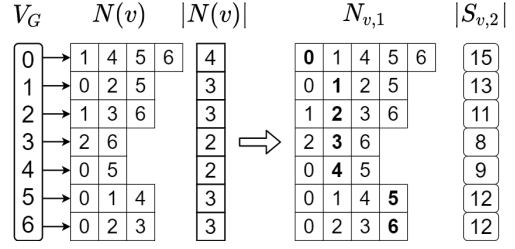


Fig. 3: An illustration for view bin packing

packing v_1 into the same bin needs $\text{size}(\{v_0, v_1\}) = 18$ due to adding $N(v_2)$ (note that $N(v_0), N(v_1)$ and $N(v_5)$ are already covered due to v_0). Thus, v_1 is put in a new bin with $\text{size}(|S_{v_1,2}|) = 13$. In this way, next-fit generates five bins $\{\{v_0\}, \{v_1\}, \{v_2, v_3\}, \{v_4, v_5\}, \{v_6\}\}$. However, the optimal packing has only three bins: $\{\{v_0, v_4, v_5\}, \{v_2, v_3, v_6\}, \{v_1\}\}$.

The next-fit algorithm is quite efficient. As the $(k - 1)$ hop neighbors of each vertex are accessed in the worst case, the time complexity is $\mathcal{O}(|V_G| \times (|E_G| / |V_G|)^{k-1})$, where $|E_G| / |V_G|$ is in the range of tens to a hundred for most large real-world graphs (e.g., those in Table II) and k ranges from 2 to 3 for most widely-used query patterns [10], [17]–[19]. However, Example 4 reveals that the effectiveness of next-fit packing highly depends on the vertex order and can incur excessive PCI-e communication overhead using a random order.

2) *K-Means-Based Vertex Ordering:* Next-fit packing is efficient but it requires a good vertex ordering to minimize the number of bins. We propose a K-means-based heuristic to generate a good vertex order. Note that given a hop number k , clustering and view bin packing only need to be done once, and we can use the pre-computed packing results to process all query patterns that have minimum $k_{\min} \leq k$. For example, for any clique pattern, the minimum $k_{\min} = 2$ and thus we only need to use 2-hop views to match a clique pattern of any size.

The K-Means model. Intuitively, more vertices can be packed together when there is more overlap among their $(k - 1)$ -hop neighbors. Therefore, we order vertices by solving a clustering problem in which each vertex v is associated with a $|V_G|$ -dimension vector X_v . The i -th position in X_v is 1 if $v_i \in N_{v,k-1}$, and 0 otherwise. A solution to this clustering problem is to use the K-Means algorithm. We can set a small cluster number (e.g., 8) in practice (details for running K-Means are given in Section V-D).

Given a cluster number $c \in \mathbb{Z}^+$, the K-Means algorithm finds a set of clusters of vertices $C = \{C_1, \dots, C_c\}$ with the following objective:

$$\arg\max_C \sum_{C_i \in C} \sum_{v \in C_i} (X_v \cdot \mu_i). \quad (2)$$

Here, μ_i is the centroid vector of C_i , which is calculated as the mean of the vectors of the vertices in C_i (following the K-Means algorithm). The count of non-zero dimensions in μ_i indicates the size of the union of the $(k - 1)$ -hop neighbors of

the vertices in C_i . In the iterative K-Means steps, a vertex v is assigned to the cluster C_i that gives the largest dot product of $X_v \cdot \mu_i$, since the $(k-1)$ -hop neighbors of v is most similar to the $(k-1)$ -hop neighbors of the vertices in this cluster.

However, the K-Means algorithm using Equation (2) does not converge, because computing the centroid by taking the mean minimizes the Euclidean distances between the centroid and the assigned vertices, but it does not maximize the dot products. To address this problem, we project the vectors onto a unit sphere and normalize X_v by setting the non-zero dimensions to $1 / \sqrt{|N_{v,k-1}|}$ instead of 1. Then, Equation (2) can be transformed to the same objective as in *Spherical K-Means* [20], which is known to have good performance for this high-dimensional data clustering:

$$\operatorname{argmax}_C \sum_{C_i \in C} \sum_{v \in C_i} \cos(X_v, \mu_i) \quad (3)$$

The transformation is deduced by cosine similarity, $\cos(a, b) = a^T b / (\|a\| \cdot \|b\|)$, where $\|a\| = \|b\| = 1$ for unit vectors. Note that the number of non-zero dimensions in μ_i still maintains the same meaning of the size of the neighbor set union. With Equation (3), the convergence is guaranteed as proved in [20], [21].

Vertex ordering. The clustering results from the K-Means algorithm only give a rough idea of which vertices should be placed near each other in the order. There are two remaining problems in order to generate a good vertex order: (1) the order among the clusters, and (2) the order of the vertices in each cluster.

For the first problem, we sort the clusters in descending order of the non-zero dimension counts of their centroids. The intuition behind this is that, since the view of a cluster is the union of the members' $(k-1)$ -hop neighbors and their adjacency lists, the non-zero dimension counts of the centroids represent the sizes of the views of the clusters. Thus, the descending order of the non-zero dimension counts greedily puts larger cluster views first for packing, thus reducing fragmentation in packing and hence also the number of bins.

For the second problem, while an arbitrary vertex order can be used in each cluster, we can further refine the vertex order using the centroid-based sorting method by recursively conducting K-Means clustering within each cluster. Recursive clustering takes more time but benefits view bin packing by improving the vertex order.

Complexity. The K-Means algorithm runs in iterations until a stopping criterion is satisfied. Assuming that the algorithm stops after t iterations and produces c clusters, the time complexity is $\mathcal{O}(tc|V_G|^2)$ and the space complexity $\mathcal{O}(|V_G|^2)$.

Enhancements. We further improve the K-Means model by two techniques.

First, for large data graphs (e.g., exceeding GPU memory), it is expensive to store $|V_G|$ dimensional vectors for $|V_G|$ vertices. Instead of $N_{v,k-1}$, we use $N_{v,1}$ to construct the vector X_v for each vertex v and represent the vectors in the sparse vector format. This reduces both the time and space complexity, i.e., the time complexity becomes $\mathcal{O}(tc|E_G|)$.

We show in Section V-D that using $N_{v,1}$ still achieves good performance for large graphs.

Second, since the K-Means algorithm is sensitive to random centroid initialization, we adapt K-Means++ [22] to improve the clustering quality. As we use cosine similarity instead of Euclidean distance used in the standard version, the original method needs to be adjusted. Following centroid initialization in K-Means++, a vertex vector is randomly chosen as the first centroid. For the remaining $(c-1)$ centroids, we initialize each centroid μ_i sequentially by choosing from the rest of the vertex vectors the one that is the least similar to the set of already chosen centroids $C_\mu = \{\mu_1, \dots, \mu_{i-1}\}$. Here, the similarity of a vertex vector X_v to C_μ is given by the cosine similarity of X_v to the most similar centroid in C_μ , i.e. $\max_{\mu_j \in C_\mu} \cos(X_v, \mu_j)$.

A challenge in applying K-Means++ is that for a set of chosen centroids, there are too many vertex vectors that have the same minimum similarity 0 (as they are orthogonal to all chosen centroids). This is typical for large graphs because the vertex vectors are sparse and most of them are orthogonal to each other. To choose a reasonable centroid from a large number of vectors, we give a high probability to the vectors of high-degree vertices. Note that we do not always select the vertex with the highest degree, because otherwise it may attract many vertices to the same cluster during K-Means iterations, resulting in a large cluster that should have been divided. Empirically, giving a higher probability to a higher-degree vertex performs better than directly using the vector of the highest degree vertex for centroid initialization.

IV. THE VSGM FRAMEWORK

We design and implement the **VSGM** framework to construct and use k -hop views for subgraph matching. VSGM supports pipelined execution to overlap the inter-dependent computation and data transfer tasks, i.e., construct a bin of views, transfer it to the GPU device and conduct subgraph matching with the views. VSGM also provides a load balancing technique in the multi-GPU setting for handling view bins with irregular computing workloads.

Preprocessing. VSGM pre-computes the set of view bins for specified hop numbers, using the method in Section III. The maximum size of a view bin is set by $\lambda - \theta$, where λ is the size of GPU memory and θ is the memory reserved for intermediate results. Since the number of bins can be large, generating all bins at once incurs high storage overhead. Instead, we store a bin ID for each vertex in the data graph, but the physical k -hop views (in the CSR format) of the vertices in each bin are constructed on-the-fly during query processing. Specifically, in each physical bin, we keep the row pointers of all vertices of the data graph (in the CSR format), but only vertices that are within $(k-1)$ hops of the source vertices can have the non-null pointers to the columns (i.e., their adjacency lists). This method avoids mapping/remapping the vertex IDs.

System architecture. As depicted in Figure 4, VSGM is composed of three major components: **Query Optimizer**,

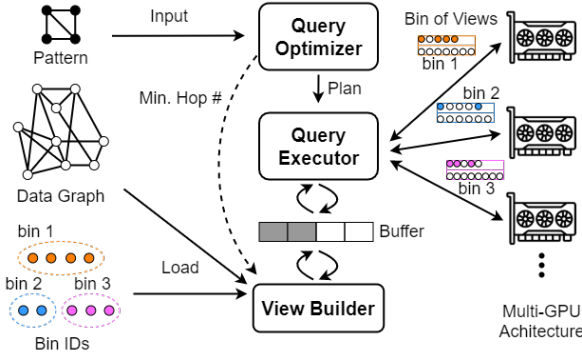


Fig. 4: The VSGM framework

Query Executor and View Builder. *Query Optimizer* computes the minimum hop number (i.e., k_{min}) and a corresponding match order for an input query pattern P (Section III-B). Then, the execution plan for matching P is constructed based on the match order and sent to *Query Executor*. *View Builder* pre-loads the data graph into the host memory. During query processing, it waits for the minimum hop number of a submitted query from *Query Optimizer*, and then constructs the corresponding view bins using the bin IDs. Meanwhile, *Query Executor* transfers each bin of views to a GPU device to conduct the execution plan on the views.

In the execution of each bin on a GPU, we use the same subgraph matching algorithm as the one used in PBE’s intra-partition search [10], which is a DFS-based method that finds matches in batches. We start each matching from a source vertex in the bin and then expand the matching from the source vertex to match other query vertices according to the given match order, where the matching starting from each source vertex in the bin is executed in parallel.

Pipelined execution. VSGM adopts a Multi-Producer-Multi-Consumer (MPMC) design to construct and consume the k -hop views of each bin on-the-fly for processing a query. A FIFO bin queue is maintained between *View Builder* and *Query Executor*. *View Builder* creates multiple producers and each producer first checks if there is an available slot in the queue, if yes, it occupies the slot and constructs a new bin of views. *Query Executor* creates one or multiple consumers which are assigned to a GPU. Each consumer checks the queue to see if any slot is filled, if yes, the consumer transfers the views in it to the GPU for matching the query pattern and then release the slot to producers.

Load balancing among GPUs. To process large data graphs or complex patterns that require more computation power, VSGM supports a multi-GPU architecture with the MPMC design. In the multi-GPU setting, we need to deal with load balancing among GPUs since different view bins lead to different subgraph matching time due to the irregularity in graph structure.

It is challenging to address load balancing while minimizing the number of bins, and a sophisticated method may also incur extra overhead. We use a simple but effective technique as

follows. As subgraph matching starting from a set of source vertices in a bin generates many search trees to find the candidates for matching each query vertex, we may estimate the matching cost of using a bin as the size of all trees. It is difficult to estimate the size of a whole search tree, but it is easy to know the size of the first layer of the tree, which is equal to the degree of the source vertex (i.e., the root of the tree). The matching cost of using a bin is then estimated as the sum of the degrees of the source vertices in the bin. VSGM uses a greedy approach to schedule view bins in descending order of their estimated matching cost. In this way, bins with heavier computation are taken by consumers earlier, which makes it easier for bins with lighter computation to smooth out the differences in the loads among GPUs.

Limitations. VSGM is sensitive to the hop number k of an input pattern. When k is large, e.g., $k > 3$, the time complexity becomes high because it has to traverse $(k - 1)$ -hop neighbors of each source vertex during bin packing. Also, k affects the size of a view. When a data graph has some very high-degree vertices, the size of their k -hop views could exceed the capacity of GPU memory. In this extreme case, the current view bin packing will fail. However, given the capacity of modern GPU memory, such extreme cases rarely happen (also because patterns with $k > 3$ are rarely used in practice). We also remark that for processing patterns with $k \geq 3$, other existing solutions would likely require even more computing resources and much longer time than VSGM (e.g., for processing query pattern P13 as reported in Section V-C). In fact, we could simply use an inter-partition method just for the extremely high-degree vertices whose k -hop views could not fit GPU memory, and this strategy would still use much less on-demand data loading than PBE.

V. PERFORMANCE EVALUATION

We evaluate the performance of VSGM against two baselines: PBE [10] and GraphPi [18]. We compare with PBE as it is the state-of-the-art GPU solution and the only known one that can handle large graphs whose size exceeds the GPU memory capacity. We also report the performance of GraphPi because it is the state-of-the-art CPU-based subgraph matching system that outperforms other CPU-based solutions.

A. Experimental Setup

Environments. The experiments for VSGM and PBE are conducted in two workstations as listed below.

- *Local server* is a machine with two Intel Xeon Silver 4114 CPU (each has 10 physical cores with hyperthreading) and 256 GB main memory. It is equipped with 4 NVIDIA GeForce RTX 2080 Ti GPUs (each has ~ 10.76 GB device memory).
- *Cloud server* is an Amazon EC2 instance (g4dn.metal) with 96 vcores and 384 GB main memory. It is equipped with 8 NVIDIA Tesla T4 GPUs (each has ~ 14.75 GB device memory).

G	$ V_G $	$ E_G $	size(GB)	avg-deg	max-deg
<i>gsh-host-2015</i> (GH)	68.7M	3.0B	11.7	43.8	2,424,071
<i>sk-2005</i> (SK)	50.6M	3.6B	13.9	71.5	8,563,816
<i>friendster</i> (FR)	65.6M	3.6B	14.0	55.1	5,214
<i>uk-union</i> (UK)	131.6M	9.3B	35.7	70.9	6,366,528
<i>datagen-sf10k-fb</i> (DG)	98.7M	18.8B	70.8	190.5	4,485

TABLE II: Data graphs used in the experiments

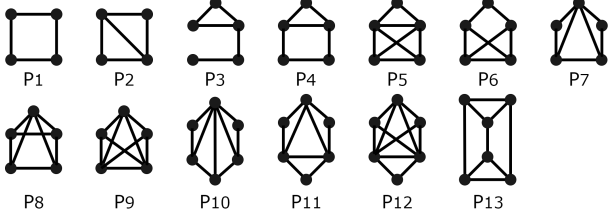


Fig. 5: Commonly used query patterns

As GraphPi is a CPU-based solution, we use CPU-only servers each with two Intel Silver 4110 2.1GHz 8-core (total 32 hyperthreads) and 128GB main memory.

All GPU programs are compiled with CUDA-11.3, and all CPU programs with GCC-7.5.0. For both compilers, we enable the `-O3` optimization level. We set the CPU parallelism for all solutions to 32 threads unless otherwise specified.

Graph datasets. We conduct experiments on data graphs with billion-scale edges. Table II lists some statistics of the data graphs, where we show the vertex/edge numbers, the size of a graph in the CSR format, and the average and maximum vertex degree of each graph. The first four graphs are real-world large-scale graphs popularly used in the performance evaluation of many graph processing systems, which cover social network [23] and web graph [24], [25]. We also use a synthetic graph *datagen-sf10k-fb* (DG) [26] generated using the LDBC benchmark [27] with scale factor 10k.

Queries. We use two classes of query patterns in the experiments. The first class consists of cliques with n vertices, where n varies from 3 to 7. n -cliques are common workloads for subgraph matching [10], [17], [28]. The second class consists of 13 patterns that contains four to six vertices, as depicted in Figure 5. They are also commonly used in existing works for experimental evaluation of subgraph matching [10], [17]–[19].

Measurements. We measure the execution time of VSGM by excluding the loading time of the data graph and bin IDs, and only include the time for bin construction, bin transfer, and matching. The execution time of PBE also excludes the time for data graph loading and partition preparation.

B. Evaluation on a Single GPU

We first evaluate the performance of VSGM on a single GPU. We show that VSGM significantly outperforms the state-of-the-art solutions [10], [18]. A key observation for the improved performance is that VSGM achieves high GPU utilization while maintaining a low overhead for using view

G	Query	VSGM	PBE (Intra+Inter)
FR	Preprocessing	7 Bins	3 Partitions
	3-cliques	60.79	393.44 (36.62+356.82)
	4-cliques	201.45	906.12 (115.07+791.05)
	5-cliques	615.01	1,284.51 (391.17+893.34)
	6-cliques	1,827.79	2,720.45 (1,225.57+1,494.88)
	7-cliques	6,365.76	14,851.81 (3,949.89+10,901.92)
SK	Preprocessing	2 Bins	2 Partitions
	3-cliques	34.13	356.75 (32.87+323.88)
	4-cliques	12,455.17	14,921.85 (12,694.23+2,227.62)
GH	Preprocessing	2 Bins	2 Partitions
	3-cliques	116.29	603.34 (108.52+494.82)

TABLE III: Execution time (sec) for matching cliques

bins. All experiments in this subsection were conducted on the local server unless otherwise stated.

Comparison using clique patterns. Table III compares the clique counting performance of VSGM with PBE using three data graphs (FR, SK, and GH), whose sizes are greater than the size of GPU memory in the local server. For preprocessing in PBE, we set the partition number to 3, 2 and 2 for FR, SK and GH, to ensure that each partition fits in GPU memory. These are the minimum numbers of partitions obtained for PBE using FENNEL [29], which is an efficient edge-cut based graph partitioning algorithm used by PBE for processing large-scale graphs. The proportions of inter-partition edges are 8.0%, 9.1% and 12.0% for FR, SK and GH. For preprocessing in VSGM, we compute the number of view bins to be 7, 2 and 2 for FR, SK and GH, where we use the (recursive) clustering setting that produced the least number of bins under the maximum bin capacity of 10GB (preprocessing results and clustering strategy are discussed in Section V-D). The queue size for on-the-fly view bin construction and also the number of producers are 3. We only use 1 consumer as only 1 GPU is used.

Table III shows that VSGM achieves $1.2\times$ (4-cliques on SK) to $10.45\times$ (3-cliques on SK) speedup compared with PBE. The performance gap between VSGM and PBE increases with the clique size because with more edges in a pattern, PBE needs more on-demand data loading for inter-partition search. In contrast, VSGM only needs to use 2-hop views to match a clique and thus the amount of data transfer is the same for matching any of the cliques.

Comparison using commonly used patterns. To further analyze the performance of VSGM, we run 10 query patterns (P1 to P10) on FR. We compare VSGM with PBE, using the same configurations as in clique counting. Note that these queries, as well as their match order for intra-partition search, are hard-coded in PBE. We examined their code and found that some of the queries (i.e., P2, P3, P6, P7 and P9) use the same match orders as the ones generated by our method with minimum k . For inter-partition search, PBE matches the queries using multiple match orders because PBE generates them to enable shared execution [10]. As our method eliminates inter-partition search, we do not need to consider such a strategy.

Figure 6 shows that VSGM achieves $4.2\times$ to $15.3\times$ speedup compared with PBE, and Table IV lists the speedup for each

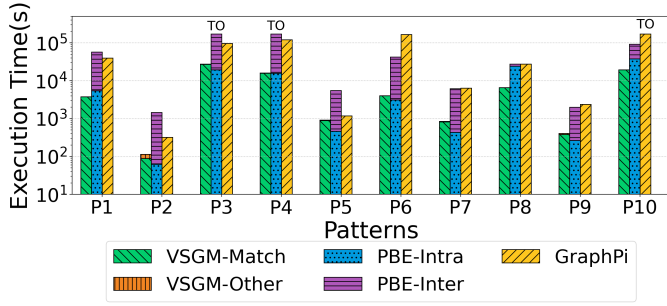


Fig. 6: Execution time (sec) of matching P1-P10 on FR (VSGM and PBE use 1 GPU in the local server, GraphPi uses all 32 hyperthreads in one CPU-only server, ‘TO’ means time-out in 48 hours)

Patterns	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
Against PBE	15.3	12.7	TO	TO	6.0	10.6	7.2	4.2	4.9	4.8
Against GraphPi	10.5	2.8	3.5	7.6	1.3	41.1	7.5	4.1	5.8	TO

TABLE IV: Speedups($\times$) against PBE and GraphPi for the query patterns in Figure 6 (‘TO’ means time-out in 48 hours)

query. PBE runs beyond time-out (48 hours) for processing P3 and P4. We also report the performance of GraphPi (using all 32 hyperthreads in a CPU-only server) as a reference to show that GPU acceleration does gain an advantage over the most competitive CPU solution, as VSGM achieves significant speedups over GraphPi for all queries except P5. GraphPi also runs beyond time-out for the more complex pattern P10.

We break down the execution time for VSGM and PBE to better understand their performance. The execution time of PBE consists of intra-partition search (**PBE-Intra**) and inter-partition search (**PBE-Inter**). As depicted in Figure 6, the execution time of PBE is dominated by inter-partition search (note that the y-axis is in log scale), although the computation for inter-partition search is much less than that for intra-partition search¹. In particular, for P3, PBE only finishes processing less than 30% of the inter-partition edges at the point of time-out. The results verify that the communication overhead due to on-demand data loading is the performance bottleneck of PBE.

The execution time of VSGM consists of the matching time (**VSGM-Match**) on GPU and the preparation time (**VSGM-Other**) which is spent on bin construction and data transfer to GPU that do not overlap with GPU computation. The preparation time for which GPU needs to wait accounts for a very small portion in the execution time, which shows that the pipelined design in VSGM is effective for hiding the data preparation and PCI-e communication overhead. Comparing VSGM’s execution time and PBE’s intra-partition search time, we can see that they are of the same order of magnitude. This again shows that PBE’s performance bottleneck is inter-partition search, and VSGM’s speedup over PBE primarily comes from the elimination of on-demand data loading. For

¹It is reported that the intra-partition matching results are much more than the inter-partition matching results [10].

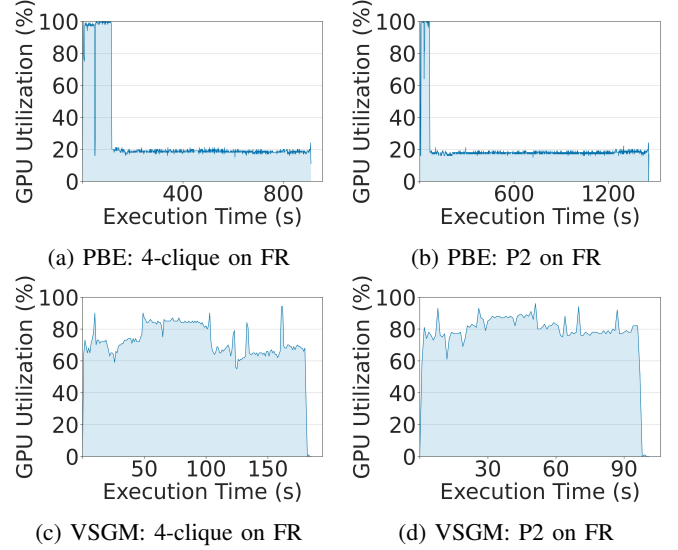


Fig. 7: Comparison of GPU utilization

P1, P8 and P10, VSGM spends even less time than PBE’s intra-partition search, because VSGM uses different match orders from PBE as VSGM picks the match orders that give the minimum k for matching the patterns.

Comparison of GPU utilization. We measure the GPU utilization of PBE and VSGM to further demonstrate that VSGM effectively improves GPU utilization by avoiding on-demand data loading. We use *NVIDIA System Management Interface*² to capture the GPU utilization at every second. Figure 7 visualizes the profiling results for two 4-vertex queries, 4-clique and P2 (the GPU utilizations for other queries are similarly high and omitted due to space limitation).

For PBE, the GPU utilization drops significantly when the computation switches from intra-partition search to inter-partition search. The average GPU utilization of PBE is 28% and 21% for 4-clique and P2. The GPU is idle most of the time, waiting for data to be transferred from the host memory for query processing in inter-partition search. On the contrary, VSGM maintains high GPU utilization throughout query execution, and achieves 71% and 77% GPU utilization on average for 4-clique and P2. There is no obvious GPU idle time (more than one second) in Figures 7c and 7d, because VSGM eliminates on-demand data loading and effectively overlaps view bin preparation and GPU computation.

Efficiency of view-based subgraph matching. We evaluate the efficiency of view-based subgraph matching by comparing VSGM’s performances in two settings: (1) when GPU memory is sufficient for storing the entire data graph and (2) when it is insufficient. We run the clique patterns on FR (14.0GB) using a single GPU (14.75GB) on the cloud server, which has GPU memory large enough to store FR. For setting (1), we use the full GPU memory capacity for processing the queries. For setting (2), we limit the maximum GPU memory usage

²<https://developer.nvidia.com/nvidia-system-management-interface>

G	Query	Limited Memory	Full Memory
FR	3-cliques	79.63	57.53
	4-cliques	391.64	388.08
	5-cliques	1,236.76	1,266.60
	6-cliques	3,701.11	3,833.95
	7-cliques	13,190.82	13,485.84

TABLE V: Execution time (sec) for matching n -clique queries with sufficient and insufficient GPU memory

(thus also limiting the bin size) to 10.76GB (the same as the GPU memory capacity on the local server) by wrapping `cudaMalloc`. The number of bins for FR is 7 for setting (2) as in the first experiment.

Table V shows that VSGM has similar performances for settings (1) and (2), even though setting (2) needs to construct the view bins on-the-fly and transfer them to GPU memory for matching the queries. This result implies that the overhead of using view bins for subgraph matching is low, as VSGM effectively overlaps view construction/transfer with GPU computation and there is no other visible overhead. Table V also shows that for the three larger patterns (i.e., 5-cliques, 6-cliques, and 7-cliques), VSGM with limited GPU memory even outperforms VSGM with sufficient GPU memory. This is attributed to both the low overhead of subgraph matching by view bins as well as the fact that the k -hop views (though simply adjacency lists) pack closely connected vertices closer to each other and hence provide better data locality. The benefit of better data locality outweighs the overhead of subgraph matching by view bins in the processing of more complex query patterns. In summary, the finding in this experiment is significant because it means that we can actually achieve very competitive performance with limited GPU memory, which is the major limitation of most existing GPU-based solutions for processing large graphs [6]–[9], [11].

C. Evaluation on Multiple GPUs

We evaluate the performance of VSGM on multiple GPUs in this experiment. We show that VSGM can process larger data graphs and more complex patterns by effectively parallelizing query execution among multiple GPUs.

Comparison using larger graphs. Figure 8 compares the performance of VSGM and PBE on two larger data graphs, UK and DG. We only list the query patterns (those that do not have heavy computation load) in Figure 8 as PBE runs time-out (i.e., 48 hours) on the other queries on these two larger data graphs. We use 4 GPUs in the cloud server as they have more capacity than the local server’s GPU. For PBE, the number of partitions for the graphs are minimum multiples of the number of GPUs (for better load balancing, i.e., this allows the partitions to be evenly distributed among the GPUs). We use 4 and 8 partitions for UK and DG. For VSGM, we set the bin capacity to 10 GB and 13 GB for UK and DG, respectively, and the number of 2-hop view bins is 7 and 105. Similar to the observation in Section V-B, the performance of VSGM is much better than PBE and the execution time of VSGM

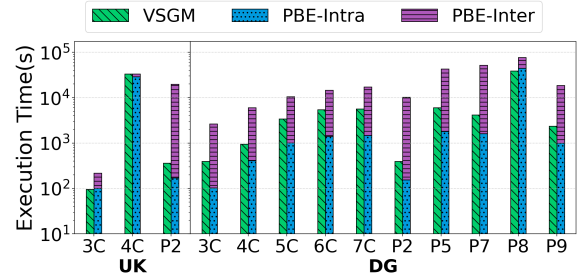


Fig. 8: Performance on larger graphs (the x-axis is for patterns, nC are n -clique patterns, P_i are the commonly used patterns)

is slightly more than the intra-partition search time of PBE, except for matching 4-cliques on UK.

We also observe from Figure 8 that PBE achieves similar overall performance for processing 4-clique and P2, but their proportions of intra-partition and inter-partition search time are radically different. On the one hand, 4-clique requires much more intra-partition computation than P2 does since 4-clique has more edges to match than P2. On the other hand, 4-clique is highly symmetric with all edges being equivalent topologically, while P2 has two groups of such equivalent edges. As a result, PBE needs to handle each group separately in one inter-partition search process, thus incurring more on-demand data loading for processing P2. The result again demonstrates the importance of eliminating on-demand data loading in VSGM.

Comparison using complex patterns. We compare the performance of VSGM and PBE for running the more complex query patterns P11 and P12 on FR. We use 4 GPUs in the local server. The bin capacity is set to 8 GB as the six-vertex patterns require more space for holding intermediate results. The number of bins is 19. Compared with PBE, VSGM achieves $3.92\times$ speedup for P11 and $3.00\times$ speedup for P12.

We also compare the performance of VSGM and PBE for matching P13 which requires 3-hop views. We use 8 GPUs in the cloud server because PBE ran out of the 256GB CPU memory in the local server for on-demand data preparation. Even using the cloud server, PBE could not finish processing P13 within 48 hours using 8 GPUs (from the statistics we collected, PBE only finished a bit more than half of the workload but was already exhausting the 384GB CPU memory for on-demand data preparation). In contrast, VSGM finishes processing P13 in 131,258.90 seconds. As a reference, we also use GraphPi to process P13 on eight CPU servers (using 8×32 hyperthreads) and it only processed a small portion of the tasks in 48 hours (from the statistics collected, our estimation is that it would take at least an order to two orders of magnitude of VSGM’s time to finish).

Effectiveness of load balancing. We break down the execution time with load balancing (**with LB**) and compare it with the break-down time by disabling load balancing in VSGM (**without LB**). Each colored chunk in Figure 9 represents the time for processing one bin when matching P11 on FR. The result shows that the estimation of the matching cost of



Fig. 9: Execution time break-down for P11 on FR (the x-axis indicates the execution time)

Vertex Ordering				Maximum Bin Size		
				10GB	8GB	6GB
Naive				70	153	318
BFS				94	178	329
K-Means	4	Random	X	11	36	105
	4	Random	O	10	28	86
	4	KMeans++	X	8	23	77
	4	KMeans++	O	7	23	79
	4 ²	KMeans++	X	8	20	52
	4 ²	KMeans++	O	8	19	51

TABLE VI: The number of bins for packing 2-hop views on FR using different vertex orders (the settings of K-Means-based vertex ordering as discussed in Section III-C2: $c = 4$, 4² means running K-Means recursively for 2 times, “Random” and “KMeans++” are the initialization methods, ‘O’/‘X’ means cluster ordering is enabled/disabled)

using each bin is effective since the computation-intensive bins are effectively prioritized and the loads among the GPUs are balanced. In fact, the current assignment by estimated costs achieves load balancing as good as assigning the bins to the GPUs by their actual processing time (i.e., the length of the chunks). From Figure 9, load balancing reduces the elapsed execution time by 21.8%.

D. Performance of K-Means-based View Bin Packing

We now evaluate the performance of our view bin packing solution. We demonstrate the effectiveness of reducing the number of bins and show its impact on the matching time.

Reducing the number of bins. We compute the number of bins for packing 2-hop views on FR using different vertex orders. As reported in Table VI, there are eight vertex orders tested (Naive, BFS, and six orders using K-Means) and three different maximum bin sizes (10GB, 8GB and 6GB) are used. The Naive ordering simply follows the ascending vertex IDs (actually a random ordering also produces similar results). The BFS ordering repeatedly picks an un-visited vertex v and all v ’s 2-hop un-visited neighbors to be ordered as a group (as to preserve data locality for matching as we are packing the 2-hop views here), where neighbors in the same group are ordered according to the order they are visited by BFS from v . For K-Means clustering, there are six kinds of vertex orders generated by different settings (as discussed in Section III-C2), i.e., recursive clustering, initialization and cluster ordering.

Bins	Query Patterns			
	3 Cliques	4 Cliques	P2	P9
70	142.6 (27.3)	274.9 (186.7)	185.3 (97.9)	477.7 (393.2)
7	57.9 (23.7)	197.3 (170.6)	114.3 (87.2)	395.3 (370.7)

TABLE VII: Total execution time (GPU matching time) in seconds on FR using 70 and 7 bins

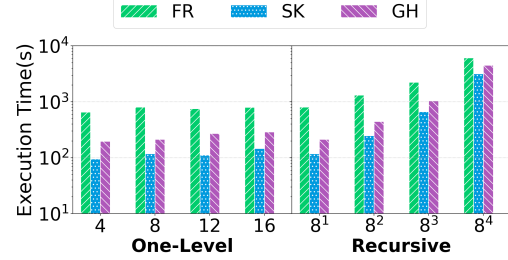


Fig. 10: Running time (sec) for K-Means (the numbers on x-axis indicate the c setting, the bars report the running time for only clustering)

Table VI shows that the Naive and BFS approaches do not reduce the number of bins, while our K-Means-based vertex ordering effectively reduces the number of bins. Among the K-Means-based ordering methods, the initialization method also affects the number of bins. The recursive clustering is effective because of the refinement it does. Enabling cluster ordering also helps reduce the number of bins but the effect is not obvious especially when “KMeans++” is used as the initialization method.

Execution time using different number of bins. We compare the execution time of VSGM for 4 patterns using 70 bins and 7 bins, which are taken from the results of Table VI. Table VII shows that using more bins takes significantly more total execution time than using few bins. Note that having more bins has very small effect on the GPU matching time, but rather increases the bin construction and transfer costs significantly, thus leading to a much longer execution time.

Detailed analysis of choosing the number of clusters. We vary the number of clusters with $c \in \{4, 8, 12, 16\}$. We also set $c = 8$ and run K-Means recursively for 1 to 4 times, resulting in 8¹ to 8⁴ clusters in total. The bin capacity is set to 10 GB. Note that view bin packing is a preprocessing using CPU only.

We first measure the efficiency of K-Means clustering and next-fit bin packing with different settings of c and recursion. Figure 10 shows that the running time of K-Means is stable for different c , while it grows almost linearly with the number of recursions. Figure 11 reports the results of packing 2-hop views using the orders acquired by the above K-Means settings, while Figures 12 and 13 reports the results for packing 3-hop views. The cluster number 0 on the x -axis means that the original vertex order is used in next-fit. The bars in Figures 11-13 report the running time of next-fit. The running time of next-fit for 2-hop views using K-Means clustering is much smaller than the execution time of the

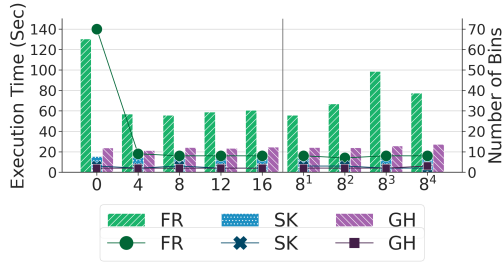


Fig. 11: 2-hop view bin packing (the numbers on x-axis indicate the c setting, the dots on the line curves show the number of bins, the bars report the running time for next-fit bin packing)

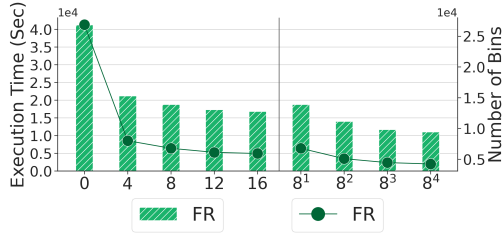


Fig. 12: 3-hop view bin packing on FR (same setting as Fig 11)

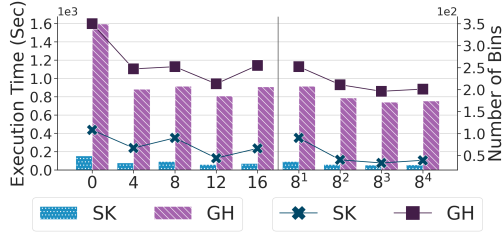


Fig. 13: 3-hop view bin packing on SK and GH (same setting as Fig 11)

corresponding queries (n -cliques and P1-P12) on all three data graphs, and for 3-hop views the running time is also one order of magnitude smaller than the execution time of P13. Considering that subgraph matching by its nature is a highly expensive graph analytics workload and the one-time view bin packing can significantly reduce the matching time of many patterns, we can conclude that this preprocessing cost is well justified. We also note that next-fit only uses 1 CPU thread.

Next, we analyze the effectiveness of using K-Means for view bin packing and discuss the strategy for configuring K-Means. We first discuss 2-hop view bin packing, which can already be used to process most commonly used query patterns (e.g., all clique patterns, P1-P12), as queries requiring more hops for views are often excessively expensive. The dots on the line curves in Figure 11 show the number of bins. Compared with using the original vertex order, ordering by K-Means effectively reduces the bin number by over 90% for FR, but makes no improvement for SK and GH. This is because FR consists of many small communities which are favorable to clustering. In contrast, SK and GH are small-world graphs

(since they have some very high-degree vertices that connect many vertices within 2 hops), and thus many vertices have highly overlapped 2-hop neighbors, which makes the views easy to pack even without clustering. We also observe that for packing 2-hop views, computing a small number of clusters (e.g., 4 or 8) is sufficiently good. Thus, we use a small cluster number in practice for packing 2-hop views.

For 3-hop view bin packing, vertex ordering by K-Means effectively reduces the number of bins for all three graphs as shown in Figures 12 and 13. Compared to simply increasing c , recursive clustering is more effective in reducing the number of bins (and also next-fit running time), which allows us to trade off clustering time for both packing efficiency and effectiveness. Note that for SK and GH, applying K-Means recursively for four times becomes worse than three times because empty clusters start to be generated. In practice, for packing views with more than 2 hops, we run K-Means recursively until empty clusters are generated.

VI. RELATED WORK

We have discussed GPU-based solutions [7]–[11] in Section II-A. Other closely related works include CPU-based solutions and general-purpose graph mining systems.

CPU-based subgraph matching. Subgraph matching for unlabeled graphs focus on execution pipeline and system-level optimizations. CWOJ [30] proposes a cost model to generate execution plans by combining binary join and worst-case optimal join. CBF [31] proposes a vertex-cover-based approach to compress the results of subgraph matching and designs a crystal decomposition based method to generate matches with optimal compression ratio. GraphZero [19] uses group theory to break symmetry in a query for automorphism elimination. GraphPi [18] designs a 2-cycle-based automorphism elimination algorithm and implements a computation reuse mechanism for query vertices sharing neighbors. There are also many subgraph matching algorithms for labeled graphs [32]–[35] following Ullmann’s backtracking-based solution [36]. Most of them adopt a two-phase approach: (1) candidate generation and auxiliary data structure construction, and (2) backtracking on the auxiliary data structure. The labeled solutions lack effective optimizations for matching unlabeled patterns. A very recent work, Circinus [37], proposes compression-based subgraph matching to reduce repeated computation during DFS-based execution, which works for both unlabeled and labeled graphs.

General-purpose graph mining systems. Some graph mining systems also support subgraph matching. Pangolin [38] supports subgraph matching on both CPU and GPU with an efficient data structure for intermediate results to exploit locality and to avoid costly materialization. Pangolin assumes that data graphs can fit in GPU memory. Many CPU-based graph mining systems have been proposed [17], [28], [39]–[44], which adopt BFS-like execution and lead to excessively high memory usage. To address this problem, Automine [28], Fractal [44] and Peregrine [17] adopt DFS-like execution and make use of graph automorphism elimination to reduce

redundant computation. These systems are all outperformed by GraphPi [18] for subgraph matching, while VSGM outperforms GraphPi by effective GPU acceleration. There are also some other graph mining systems [45], [46] and graph processing systems [47] that can be used to process subgraph matching, but they require users to implement the matching logic of each query pattern. They are also CPU-based systems and thus the parallelism in processing subgraph matching is limited compared with GPU-based solutions.

VII. CONCLUSIONS

We propose VSGM, a multi-GPU framework for subgraph matching on large graphs. VSGM addresses the problem of on-demand data loading with a view-based solution. Our comprehensive performance evaluation verifies that VSGM significantly outperforms the state-of-the-art GPU solution PBE [10] with its view-based solution, thereby demonstrating the significance of eliminating on-demand data loading in GPU-based subgraph matching on graphs that cannot fit in GPU memory. VSGM also effectively overlaps view bin construction/transfer with GPU computation, thus achieving high GPU utilization and low overhead of using view bins for subgraph matching with limited GPU memory. We also show that VSGM gains significant speedup over the fastest CPU-based solution, GraphPi [18], with GPU acceleration.

ACKNOWLEDGMENT

The authors would like to thank the reviewers for their valuable comments and suggestions that have helped significantly improve the paper.

REFERENCES

- [1] S. Sahu, A. Mhedhbi, S. Salihoglu, J. Lin, and M. T. Özsu, "The ubiquity of large graphs and surprising challenges of graph processing," *Proc. VLDB Endow.*, vol. 11, no. 4, pp. 420–431, 2017. [Online]. Available: <http://www.vldb.org/pvldb/vol11/p420-sahu.pdf>
- [2] T. A. B. Snijders, P. E. Pattison, G. L. Robins, and M. S. Handcock, "New specifications for exponential random graph models," *Sociological Methodology*, vol. 36, no. 1, pp. 99–153, 2006. [Online]. Available: <https://doi.org/10.1111/j.1467-9531.2006.00176.x>
- [3] B. Chandramouli, J. Goldstein, and D. Maier, "High-performance dynamic pattern matching over disordered streams," *Proc. VLDB Endow.*, vol. 3, no. 1, pp. 220–231, 2010. [Online]. Available: http://www.vldb.org/pvldb/vldb2010/pvldb_vol3/R19.pdf
- [4] N. Pržulj, D. G. Corneil, and I. Jurisica, "Efficient estimation of graphlet frequency distributions in protein-protein interaction networks," *Bioinformatics*, vol. 22, no. 8, p. 974–980, Apr. 2006. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btl030>
- [5] X. Yan, P. S. Yu, and J. Han, "Graph indexing: A frequent structure-based approach," in *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '04. New York, NY, USA: Association for Computing Machinery, 2004, p. 335–346. [Online]. Available: <https://doi.org/10.1145/1007568.1007607>
- [6] W. Lin, X. Xiao, X. Xie, and X. Li, "Network motif discovery: A GPU approach," in *31st IEEE International Conference on Data Engineering, ICDE 2015, Seoul, South Korea, April 13-17, 2015*, J. Gehrke, W. Lehner, K. Shim, S. K. Cha, and G. M. Lohman, Eds. IEEE Computer Society, 2015, pp. 831–842. [Online]. Available: <https://doi.org/10.1109/ICDE.2015.7113337>
- [7] H. N. Tran, J. Kim, and B. He, "Fast subgraph matching on large graphs using graphics processors," in *Database Systems for Advanced Applications - 20th International Conference, DASFAA 2015, Hanoi, Vietnam, April 20-23, 2015, Proceedings, Part I*, ser. Lecture Notes in Computer Science, M. Renz, C. Shahabi, X. Zhou, and M. A. Cheema, Eds., vol. 9049. Springer, 2015, pp. 299–315. [Online]. Available: https://doi.org/10.1007/978-3-319-18120-2_18
- [8] L. Wang and J. D. Owens, "Fast gunrock subgraph matching (GSM) on gpus," *CoRR*, vol. abs/2003.01527, 2020. [Online]. Available: <https://arxiv.org/abs/2003.01527>
- [9] L. Zeng, L. Zou, M. T. Özsu, L. Hu, and F. Zhang, "GSI: gpu-friendly subgraph isomorphism," in *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020*. IEEE, 2020, pp. 1249–1260. [Online]. Available: <https://doi.org/10.1109/ICDE48307.2020.00112>
- [10] W. Guo, Y. Li, M. Sha, B. He, X. Xiao, and K. Tan, "Gpu-accelerated subgraph enumeration on partitioned graphs," in *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, D. Maier, R. Pottinger, A. Doan, W. Tan, A. Alawini, and H. Q. Ngo, Eds. ACM, 2020, p. 1067–1082. [Online]. Available: <https://doi.org/10.1145/3318464.3389699>
- [11] L. Xiang, A. Khan, E. Serra, M. Halappanavar, and A. Sukumaran-Rajam, "cuts: scaling subgraph isomorphism on distributed multi-gpu systems using trie based data structure," in *SC '21: The International Conference for High Performance Computing, Networking, Storage and Analysis, St. Louis, Missouri, USA, November 14 - 19, 2021*, B. R. de Supinski, M. W. Hall, and T. Gamblin, Eds. ACM, 2021, pp. 69:1–69:14. [Online]. Available: <https://doi.org/10.1145/3458817.3476214>
- [12] D. S. Johnson, "Fast algorithms for bin packing," *J. Comput. Syst. Sci.*, vol. 8, no. 3, pp. 272–314, 1974. [Online]. Available: [https://doi.org/10.1016/S0022-0000\(74\)80026-7](https://doi.org/10.1016/S0022-0000(74)80026-7)
- [13] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [14] W. Fang, B. He, Q. Luo, and N. K. Govindaraju, "Mars: Accelerating mapreduce with graphics processors," *IEEE Trans. Parallel Distributed Syst.*, vol. 22, no. 4, pp. 608–620, 2011. [Online]. Available: <https://doi.org/10.1109/TPDS.2010.158>
- [15] Y. Wang, Y. Pan, A. A. Davidson, Y. Wu, C. Yang, L. Wang, M. Osama, C. Yuan, W. Liu, A. T. Riffel, and J. D. Owens, "Gunrock: GPU graph analytics," *CoRR*, vol. abs/1701.01170, 2017. [Online]. Available: <http://arxiv.org/abs/1701.01170>
- [16] M. Sindelar, R. K. Sitaraman, and P. J. Shenoy, "Sharing-aware algorithms for virtual machine colocation," in *SPAA 2011: Proceedings of the 23rd Annual ACM Symposium on Parallelism in Algorithms and Architectures, San Jose, CA, USA, June 4-6, 2011 (Co-located with FCRC 2011)*, R. Rajaraman and F. M. auf der Heide, Eds. ACM, 2011, pp. 367–378. [Online]. Available: <https://doi.org/10.1145/1989493.1989554>
- [17] K. Jamshidi, R. Mahadasa, and K. Vora, "Peregrine: a pattern-aware graph mining system," in *EuroSys '20: Fifteenth EuroSys Conference 2020, Heraklion, Greece, April 27-30, 2020*, A. Bilas, K. Magoutis, E. P. Markatos, D. Kostic, and M. I. Seltzer, Eds. ACM, 2020, pp. 13:1–13:16. [Online]. Available: <https://doi.org/10.1145/3342195.3387548>
- [18] T. Shi, M. Zhai, Y. Xu, and J. Zhai, "Graphpi: high performance graph pattern matching through effective redundancy elimination," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, C. Cui, I. Qualters, and W. T. Kramer, Eds. IEEE/ACM, 2020, p. 100. [Online]. Available: <https://doi.org/10.1109/SC41405.2020.00104>
- [19] D. Mawhirter, S. Reinehr, C. Holmes, T. Liu, and B. Wu, "Graphzero: A high-performance subgraph matching system," *ACM SIGOPS Oper. Syst. Rev.*, vol. 55, no. 1, pp. 21–37, 2021. [Online]. Available: <https://doi.org/10.1145/3469379.3469383>
- [20] I. S. Dhillon and D. S. Modha, "Concept decompositions for large sparse text data using clustering," *Mach. Learn.*, vol. 42, no. 1/2, pp. 143–175, 2001. [Online]. Available: <https://doi.org/10.1023/A:1007612920971>
- [21] J. Peng and J. Zhu, "Refining spherical k-means for clustering documents," in *Proceedings of the International Joint Conference on Neural Networks, IJCNN 2006, part of the IEEE World Congress on Computational Intelligence, WCCI 2006, Vancouver, BC, Canada, 16-21 July 2006*. IEEE, 2006, pp. 4146–4150. [Online]. Available: <https://doi.org/10.1109/IJCNN.2006.246962>

- [22] D. Arthur and S. Vassilvitskii, “k-means++: the advantages of careful seeding,” in *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2007, New Orleans, Louisiana, USA, January 7-9, 2007*, N. Bansal, K. Pruhs, and C. Stein, Eds. SIAM, 2007, pp. 1027–1035. [Online]. Available: <http://dl.acm.org/citation.cfm?id=1283383.1283494>
- [23] J. Yang and J. Leskovec, “Defining and evaluating network communities based on ground-truth,” *Knowl. Inf. Syst.*, vol. 42, no. 1, pp. 181–213, 2015. [Online]. Available: <https://doi.org/10.1007/s10115-013-0693-z>
- [24] P. Boldi and S. Vigna, “The webgraph framework I: compression techniques,” in *Proceedings of the 13th international conference on World Wide Web, WWW 2004, New York, NY, USA, May 17-20, 2004*, S. I. Feldman, M. Uretsky, M. Najork, and C. E. Wills, Eds. ACM, 2004, pp. 595–602. [Online]. Available: <https://doi.org/10.1145/988672.988752>
- [25] P. Boldi, M. Rosa, M. Santini, and S. Vigna, “Layered label propagation: A multiresolution coordinate-free ordering for compressing social networks,” *CoRR*, vol. abs/1011.5425, 2010. [Online]. Available: <http://arxiv.org/abs/1011.5425>
- [26] A. Iosup, T. Hegeman, W. L. Ngai, S. Heldens, A. Prat-Pérez, T. Manhardt, H. Chafi, M. Capota, N. Sundaram, M. J. Anderson, I. G. Tanasz, Y. Xia, L. Nai, and P. A. Boncz, “LDBC graphalytics: A benchmark for large-scale graph analysis on parallel and distributed platforms,” *Proc. VLDB Endow.*, vol. 9, no. 13, pp. 1317–1328, 2016. [Online]. Available: <http://www.vldb.org/pvldb/vol9/p1317-iosup.pdf>
- [27] P. A. Boncz, I. Fundulaki, A. Gubichev, J. L. Larriba-Pey, and T. Neumann, “The linked data benchmark council project,” *Datenbank-Spektrum*, vol. 13, no. 2, pp. 121–129, 2013. [Online]. Available: <https://doi.org/10.1007/s13222-013-0125-y>
- [28] D. Mawhirter and B. Wu, “Automine: harmonizing high-level abstraction and high performance for graph mining,” in *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP 2019, Huntsville, ON, Canada, October 27-30, 2019*, T. Brecht and C. Williamson, Eds. ACM, 2019, pp. 509–523. [Online]. Available: <https://doi.org/10.1145/3341301.3359633>
- [29] C. E. Tsourakakis, C. Gkantsidis, B. Radunovic, and M. Vojnovic, “FENNEL: streaming graph partitioning for massive scale graphs,” in *Seventh ACM International Conference on Web Search and Data Mining, WSDM 2014, New York, NY, USA, February 24-28, 2014*, B. Carterette, F. Diaz, C. Castillo, and D. Metzler, Eds. ACM, 2014, pp. 333–342. [Online]. Available: <https://doi.org/10.1145/2556195.2556213>
- [30] A. Mhedhbi and S. Salihoglu, “Optimizing subgraph queries by combining binary and worst-case optimal joins,” *CoRR*, vol. abs/1903.02076, 2019. [Online]. Available: <http://arxiv.org/abs/1903.02076>
- [31] M. Qiao, H. Zhang, and H. Cheng, “Subgraph matching: on compression and computation,” *Proc. VLDB Endow.*, vol. 11, no. 2, pp. 176–188, 2017. [Online]. Available: <http://www.vldb.org/pvldb/vol11/p176-qiao.pdf>
- [32] W. Han, J. Lee, and J. Lee, “Turbo_{iso}: towards ultrafast and robust subgraph isomorphism search in large graph databases,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2013, New York, NY, USA, June 22-27, 2013*, K. A. Ross, D. Srivastava, and D. Papadias, Eds. ACM, 2013, pp. 337–348. [Online]. Available: <https://doi.org/10.1145/2463676.2465300>
- [33] F. Bi, L. Chang, X. Lin, L. Qin, and W. Zhang, “Efficient subgraph matching by postponing cartesian products,” in *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016, San Francisco, CA, USA, June 26 - July 01, 2016*, F. Özcan, G. Koutrika, and S. Madden, Eds. ACM, 2016, pp. 1199–1214. [Online]. Available: <https://doi.org/10.1145/2882903.2915236>
- [34] B. Bhattarai, H. Liu, and H. H. Huang, “CECI: compact embedding cluster index for scalable subgraph matching,” in *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, P. A. Boncz, S. Manegold, A. Ailamaki, A. Deshpande, and T. Kraska, Eds. ACM, 2019, pp. 1447–1462. [Online]. Available: <https://doi.org/10.1145/3299869.3300086>
- [35] M. Han, H. Kim, G. Gu, K. Park, and W. Han, “Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together,” in *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, P. A. Boncz, S. Manegold, A. Ailamaki, A. Deshpande, and T. Kraska, Eds. ACM, 2019, pp. 1429–1446. [Online]. Available: <https://doi.org/10.1145/3299869.3319880>
- [36] J. R. Ullmann, “An algorithm for subgraph isomorphism,” *J. ACM*, vol. 23, no. 1, pp. 31–42, 1976. [Online]. Available: <http://doi.acm.org/10.1145/321921.321925>
- [37] T. Jin, B. Li, Y. Li, Q. Zhou, Q. Ma, Y. Zhao, H. Chen, and J. Cheng, “Circinus: Fast redundancy-reduced subgraph matching,” 2023, in press, to appear in Proceedings of the 42nd ACM SIGMOD International Conference on Management of Data.
- [38] X. Chen, R. Dathathri, G. Gill, and K. Pingali, “Pangolin: An efficient and flexible graph mining system on CPU and GPU,” *Proc. VLDB Endow.*, vol. 13, no. 8, pp. 1190–1205, 2020. [Online]. Available: <http://www.vldb.org/pvldb/vol13/p1190-chen.pdf>
- [39] C. H. C. Teixeira, A. J. Fonseca, M. Serafini, G. Siganos, M. J. Zaki, and A. Aboulmaga, “Arabesque: a system for distributed graph mining,” in *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*, E. L. Miller and S. Hand, Eds. ACM, 2015, pp. 425–440. [Online]. Available: <https://doi.org/10.1145/2815400.2815410>
- [40] H. Chen, M. Liu, Y. Zhao, X. Yan, D. Yan, and J. Cheng, “G-miner: an efficient task-oriented graph mining system,” in *Proceedings of the Thirteenth EuroSys Conference, EuroSys 2018, Porto, Portugal, April 23-26, 2018*, R. Oliveira, P. Felber, and Y. C. Hu, Eds. ACM, 2018, pp. 32:1–32:12. [Online]. Available: <https://doi.org/10.1145/3190508.3190545>
- [41] D. Yan, G. Guo, M. M. R. Chowdhury, M. T. Özsu, W. Ku, and J. C. S. Lui, “G-thinker: A distributed framework for mining subgraphs in a big graph,” in *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020*. IEEE, 2020, pp. 1369–1380. [Online]. Available: <https://doi.org/10.1109/ICDE48307.2020.00122>
- [42] K. Wang, Z. Zuo, J. Thorpe, T. Q. Nguyen, and G. H. Xu, “Rstream: Marrying relational algebra with streaming for efficient graph mining on a single machine,” in *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*, A. C. Arpaci-Dusseau and G. Völker, Eds. USENIX Association, 2018, pp. 763–782. [Online]. Available: <https://www.usenix.org/conference/osdi18/presentation/wang>
- [43] C. Zhao, Z. Zhang, P. Xu, T. Zheng, and J. Guo, “Kaleido: An efficient out-of-core graph mining system on a single machine,” in *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020*. IEEE, 2020, pp. 673–684. [Online]. Available: <https://doi.org/10.1109/ICDE48307.2020.00064>
- [44] V. V. dos Santos Dias, C. H. C. Teixeira, D. O. Guedes, W. M. Jr., and S. Parthasarathy, “Fractal: A general-purpose graph pattern mining system,” in *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, P. A. Boncz, S. Manegold, A. Ailamaki, A. Deshpande, and T. Kraska, Eds. ACM, 2019, pp. 1357–1374. [Online]. Available: <https://doi.org/10.1145/3299869.3319875>
- [45] H. Chen, M. Liu, Y. Zhao, X. Yan, D. Yan, and J. Cheng, “G-miner: an efficient task-oriented graph mining system,” in *Proceedings of the Thirteenth EuroSys Conference, EuroSys 2018, Porto, Portugal, April 23-26, 2018*, R. Oliveira, P. Felber, and Y. C. Hu, Eds. ACM, 2018, pp. 32:1–32:12. [Online]. Available: <https://doi.org/10.1145/3190508.3190545>
- [46] H. Chen, X. Wang, C. Huang, J. Fang, Y. Hou, C. Li, and J. Cheng, “Large scale graph mining with g-miner,” in *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, P. A. Boncz, S. Manegold, A. Ailamaki, A. Deshpande, and T. Kraska, Eds. ACM, 2019, pp. 1881–1884. [Online]. Available: <https://doi.org/10.1145/3299869.3320219>
- [47] T. Jin, Z. Cai, B. Li, C. Zheng, G. Jiang, and J. Cheng, “Improving resource utilization by timely fine-grained scheduling,” in *EuroSys ’20: Fifteenth EuroSys Conference 2020, Heraklion, Greece, April 27-30, 2020*, A. Bilas, K. Magoutis, E. P. Markatos, D. Kostic, and M. I. Seltzer, Eds. ACM, 2020, pp. 20:1–20:16. [Online]. Available: <https://doi.org/10.1145/3342195.3387551>

Appendix: Artifact Description/Artifact Evaluation

SUMMARY OF THE EXPERIMENTS REPORTED

We ran two types of experiments in this paper: (1) the evaluation on the performance of subgraph matching and (2) the evaluation on the performance of view bin packing. For subgraph matching, we compared the performance of three systems: VSGM (our system), PBE (the state-of-the-art GPU solution), and GraphPi (the state-of-the-art CPU solution). The experiments can be repeated with the following steps: (1) download and prepare the graph datasets in the edge list format, (2) download and compile the three systems, (3) pre-process the data graphs, and (4) run the matching of the input query patterns with the systems and obtain the query results. For view bin packing, it first ran K-Means-based clustering on the data graphs and then conducted view bin packing. We list the relevant dependencies and other details as follows.

Hardware dependencies

- NVIDIA GeForce RTX 2080 Ti GPUs.
- NVIDIA Tesla T4 GPUs.

Software dependencies

- VSGM can be accessed at <https://github.com/kygx-legend/vsgm.git>. The commit version is *be624ce*.
- PBE can be accessed at <https://github.com/guowentian/SubgraphMatchGPU.git>. However, to align with our experiments, we use the forked repository at <https://github.com/kygx-legend/SubgraphMatchGPU.git>, which has necessary modifications, e.g., more query patterns and the same hardware configurations. The commit version is *bda8608*.
- GraphPi can be assessed at <https://github.com/thu-pacman/GraphPi.git>. We also provide a modified version at <https://github.com/kygx-legend/GraphPi.git>, which adds a specified data loader for the *friendster* graph and prepares the scripts to run the experiments of subgraph matching. The commit version is *c8c3209*.

All GPU programs are compiled with CUDA-11.3, and all CPU programs with GCC-7.5.0. For both compilers, we enable the `-O3` optimization level.

Dataset sources - All datasets are publicly available:

- *friendster* (FR). The raw data is at <https://snap.stanford.edu/data/bigdata/communities/com-friendster.unigraph.txt.gz>.
- *gsh-host-2015* (GH), *sk-2005* (SK), *uk-union* (UK). These raw data are at <https://law.di.unimi.it/datasets.php> (only *[sk-2005, gsh-host-2015, uk-union-2006-06-2007-05].{graph, properties}* files).
- *datagen-sf10k-fb* (DG). The raw data is at <https://ldbcouncil.org/benchmarks/graphalytics/>.

Except *friendster* (FR), <https://github.com/kygx-legend/gtools.git> includes the tools to extract edge list from raw data.

AUTHOR-CREATED OR MODIFIED ARTIFACTS:

Artifact 1

Persistent ID: <https://doi.org/10.5281/zenodo.6912354>

Artifact name: VSGM

Artifact 2

Persistent ID: <https://doi.org/10.5281/zenodo.6626375>

Artifact name: PBE

Artifact 3

Persistent ID: <https://doi.org/10.5281/zenodo.6622289>

Artifact name: GraphPi

Artifact 4

Persistent ID: <https://doi.org/10.5281/zenodo.6616054>

Artifact name: Raw Data Converter

Reproduction of the artifact with container: The accordingly experimental setups are described as follows.

VSGM. After downloading the source code, follow the steps in *README.md* to compile the programs. We provide the pre-built executables in **guanxian/gpu-exp** Docker container. After pulling **guanxian/gpu-exp** from Docker Hub, all pre-built executables are in `/vsgm.preprocess_graph, features, kmeans, view_packing` are for data preprocessing. *vsgm-10g*, *vsgm-14g* are the subgraph matching applications for different GPU settings (10g for 10.6G maximum memory and 14g for 14.5G maximum memory). *prepare_data.sh*, *run_query.sh* are the scripts for convenience of running the experiments. Detailed instructions are listed as follows.

```
# start docker image with specified volume and GPUs
docker run -it -v {VOLUME}:/exp --gpus {GPUS}
guanxian/gpu-exp
```

```
cp -r /vsgm . # copy all executables and scripts
cd ./vsgm # go into ./vsgm
```

```
./prepare_data.sh # download FR and preprocess
# FR_PATH=./data/com-friendster.unigraph.txt.bin
# run 3-cliques on single GPU
./run_query.sh run vsgm-10g ${FR_PATH} 10 kc3
# run all queries on single GPU
./run_query.sh run_all vsgm-10g ${FR_PATH} 10
# run all p2 on 2 GPUs
./run_query.sh run_multi vsgm-10g ${FR_PATH} 10 3 2 p2
```

PBE. After downloading the source code, follow the steps in *README.md* and *src/gpu/README.md* to compile the program *src/gpu/patterngraphgpu*. How to run a subgraph matching application is also included in the readme files. We also provide the pre-built executables in **guanxian/gpu-exp** Docker container. It includes *preprocess_graph, fennel, pbe-10g, pbe-14g, prepare_data.sh, run_query.sh* in */pbe*. Detailed instructions are listed as follows.


```
# start docker image with specified volume and GPUs
docker run -it -v {VOLUME}:/exp --gpus {GPUS}
guanxian/gpu-exp
```

```
cp -r /pbe . # copy all executables and scripts
cd ./pbe # go into ./pbe
```

```
./prepare_data.sh # download FR and preprocess
# FR_PATH=./data/com-friendster.ungraph.txt.bin
# run 3-cliques on single GPU
./run_query.sh run pbe-10g ${FR_PATH} 3 kc3
# run all queries on single GPU
./run_query.sh run_all pbe-10g ${FR_PATH} 3
# run all p2 on 2 GPUs
./run_query.sh run_multi pbe-10g ${FR_PATH} 3 2 p2
```

GraphPi. After downloading the source code on GitHub, GraphPi can be compiled with CMake. In addition, a pre-built image can be pulled from Docker Hub with repository **guanxian/cpu-exp**. Two script files `/frameworks/GraphPi/prepare_data.sh` and `/frameworks/GraphPi/run_query.sh` can be used for running the experiments. Detailed instructions are listed as follows.

```
# start docker image
docker run -it guanxian/cpu-exp

# download FR and convert the format
./prepare_data.sh
# run query 9
./run_query.sh run friendster 9 localhost
# run 10 queries
./run_query.sh run_all friendster localhost
```

For setting $kc\{3 - 7\}$, it is referring to TABLE III on FR graph. For setting $p\{1 - 10\}$, it is referring to Fig. 6 and TABLE IV.

Raw Data Converter. After downloading the source code, follow the instructions in `ldbc/README.md` and `webgraph/README.md` to extract the edge list from the data graphs. This tool is not for *friendster* (FR) because the raw data of FR is already in the edge list format.

Possible Errors.

- Size of GPU Memory - Because the maximum size of GPU memory is set before compiling. If needs to change the maximum size, we suggest to download the source code and modify the value of `kDeviceMemoryLimits` accordingly before compiling the programs.
- Error of `cnmem` for PBE - `cnmem` is a compulsory library for PBE. If any related error occurs, e.g. `CNMEM_STATUS_INVALID_ARGUMENT`, we suggest to check if `cnmem` can be built successfully and add the `libcnmem.so` to `LD_LIBRARY_PATH`.
- Error about GPU - We suggest to check the GPU architecture (GPU gencode) and make it compatible with related CUDA version.